

Module: 2

Verilog HDL

Module:2 Verilog HDL

- Lexical Conventions,
- Ports and Modules,
- Operators,
- Gate Level Modelling,
- Dataflow Modelling,
- Behavioural Modeling,
- Test Bench

Lexical Conventions

- Lexical conventions create **consistent formats** for enhancing the effectiveness and efficiency of terminology-related activities
- In Verilog, the basic unit is called **module**.
- Modules cannot contain the definitions of other module
- A Module can, however, be instantiated with in another module.
- Allows the creation of a hierarchy in a Verilog description.
- Semicolon ; Statement terminator
- // - Single line comment
- /* */ - Multi-line comment
- Case sensitive

Module

- A module is the fundamental descriptive unit in the Verilog language.
- It is declared by the keyword **module** and must always be terminated by the keyword **endmodule**
- Each statement must be terminated with a semicolon, but there is no semicolon after endmodule.

Example

```
module name (port list);  
    input a, b;  
    output f;  
    wire w1, w2;  
    functional description;  
endmodule
```

Keywords

- Verilog model is composed of keywords such as **module**, **endmodule**, **input**, **output**, **wire**, **and**, **or**, and **not**.
- Predefined identifiers to define the language
- All keywords are defined in **lower case**
- Cannot be used as identifiers

Keywords

always	endfunction	input	signed
and	endgenerate	integer	task
assign	endmodule	join	time
automatic	endprimitive	localparam	real
begin	endspecify	module	realtime
case	endtable	nand	reg
casex	endtask	negedge	unsigned
casez	event	nmos	wait
deassign	for	nor	while
default	force	not	wire
defparam	forever	or	
design	fork	output	
disable	function	parameter	
edge	generate	pmos	
else	genvar	posedge	
end	include	primitive	
endcase	initial	specify	
endconfig	inout	specparam	

Identifier

- User-provided names for Verilog objects in the descriptions
- Legal characters are “a-z”, “A-Z”, “0-9”, “_”, and “\$”
- Case sensitive
- First character has to be a letter or an “_”
- Example: Count, _R2D2, FIVE\$

Ports

- Three types of ports
 - Input (keyword - input)
 - Output (keyword – output)
 - Inout (keyword – inout)
- Port declarations example

input a;

input a, b;

input [3:0] c, d;

output [4:0] y;

inout x;

Port declaration

- The port list of a module is the interface between the module and its environment.
- The port list is enclosed in parentheses, and commas are used to separate elements of the list. The statement is terminated with a semicolon (;).
- Keywords input and output specify which of the ports are inputs and which are outputs.
- Internal connections are declared as wire (declared with the keyword wire)

Gate instance

- Predefined primitives :
 - and
 - not
 - or
- The output of a primitive gate is always listed first, followed by the inputs.
- For example,
and a1(c, x, y);
- The AND gate is represented by the and primitive, is named a1, and has output c and inputs x and y.

Gate Delays

- All physical circuits exhibit a propagation delay between the transition of an input and a resulting transition of an output.
- When an HDL model of a circuit is simulated, it is sometimes necessary to specify the amount of delay from the input to the output of its gates.

`xor #(30)x1(s, x, y); // xor gate has time delay of 30 ns`

`and #(20)a1(c, x, y); // and gate has time delay of 20 ns`

Note: If no timescale is specified, a simulator may display dimensionless values or default to a certain time unit, usually 1 ns ($=10^{-9}$ s)

Operators

Operator	Description
&	AND
~&	NAND
 	OR
~ 	NOR
^	XOR
~^ or ^~	XNOR

Types of Operators	Symbols
Arithmetic Operators	+, -, *, /, %
Relational Operators	<, <=, >, >=
Logical Equality Operators	==, !=
Case Equality Operators	===, !==
Logical Operators	!, &&,
Bit-wise Operators	~, &, , ^, ~^
Unary Reduction Operators	&, ~&, , ~ , ^, ~^
Shift Operators	<<, >>
Conditional Operators	? :
Concatenation Operators	{ }
Replication Operator	{ { } }

Data Types

- Data Types are divided into

1. NETS

2. Variables

- Net: represents the physical connection between components. e.g. wire,
- Variable: represents the storage of values in the design. e.g. 'reg' and 'integer'

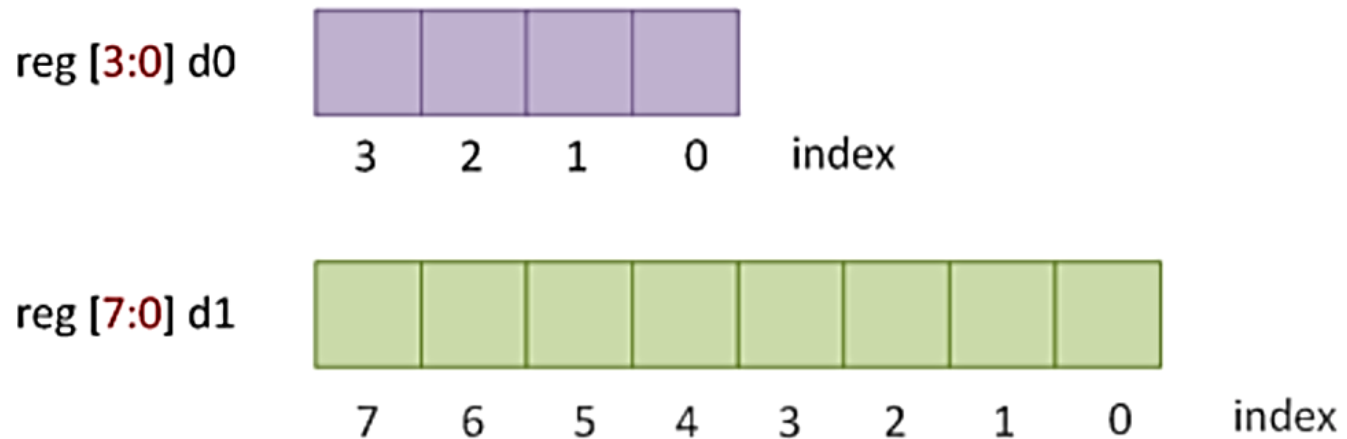
Wire

- A wire is verilog data type used to connect elements and to connect nets.
- The "wire" is similar to electrical wire that is used to connect the components on breadboard
- `wire [3:0] n0;` `// 4-bit wire -> this is a vector`



Variables

- Register variables are used in behavioral modeling, are assigned values by procedural statements and store information.



A variable on the other hand is an abstraction of a data storage element and can hold values. A flip-flop is a good example of a storage element.

- Scalar is a single value (usually one bit)
- Vector is a set of values of a given type
 - `reg [7:0] v1, v2;                   // 8-bit vectors, MSB is highest bit #`
 - `wire [1:4] v3;                       // 4-bit vector, MSB is lowest bit #`
 - `reg [31:0] memory [0:127];   // array of 128 32-bit values`
 - `{v1, v2}                    // 16-bit vector: concatenate bits/vectors into larger vector`

Number Specification

1. Sized numbers
2. Unsized numbers
3. Negative numbers
4. Unknown and
5. High-impedance values

1. Sized numbers

General syntax: <size>'<base><number>

<size> : number of bits (in decimal)

<base> :d or D for decimal (radix 10)

b or B for binary (radix 2)

o or O for octal (radix 8)

h or H for hexadecimal (radix 16)

<number> : is the number in radix <base>

- **4'b1011**
- **This is a 4-bit binary number**
- **12'hdec**
- **This is a 12-bit hexadecimal number (1101 1110 1100)**
- **16'd255**
- **This is a 16-bit binary number (00000000 11111111)**

2. Unsized numbers

- Default base is decimal
- Default size is at least 32 (depends on Verilog compiler)

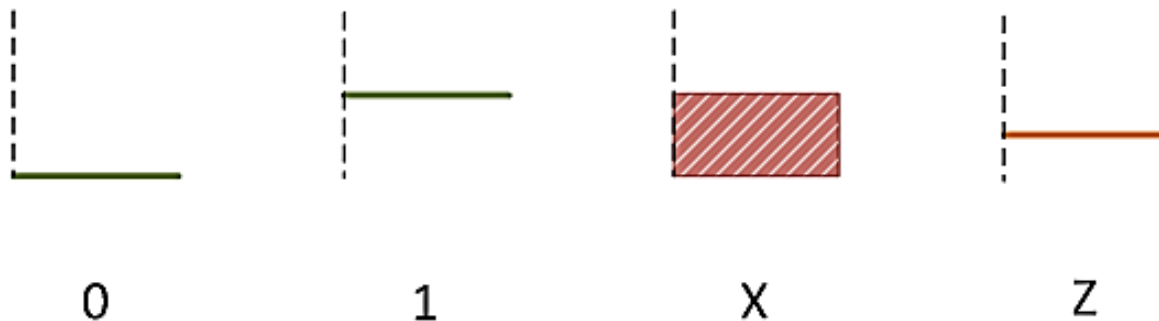
3. Negative numbers

- Put the sign before the <size>
- Two's complement is used to store the value

The Verilog HDL value set consists of four basic values:

Value	Definition
0	Logic zero or false
1	Logic one or true
x	Unknown logical value
z	High impedance of tristate gate

Z state when the output is not driven by the input(s), that means output is neither high (1) nor low (0). The output is electrically disconnected from the circuit.



- Verilog has two symbols for unknown and high impedance values. These values are important for modeling real circuits.
- **Unknown Value: denoted by x**
- **High Impedance Value: denoted by z**
- 12'h13x // This is a 12-bit hex number ; 4 least significant bits unknown (0001 0011 xxxx)
- 9'ox13 // This is a 9-bit octal number ; 3 most significant bits unknown (xxx 001 011)

- 138 // decimal number, 32 bit as
0000000000000000000000000000000010001010
- 10'd138 // decimal number, 10 bit as 0010001010
- 6'o74 // octal number, 6 bits as 111100
- 24'h25F // hexadecimal number, 24 bit as
0000000000000000000000001001011111
- 8'hxB // hexadecimal number, 8 bit as xxxx1011
- 3'b010 // binary number, 3 bits as 010
- -6'b101 // 6 bit, two's complement of 000101 or 111011
- -10'd15 // 10 bit, two's complement of 0000001111 or 1111110001
- 5'd124 // decimal number, 5 bits as 11100 since 7 bits are required
- 12'oF2 // invalid, F is not an octal digit

HDL models of combinational circuits

- The logic of a module can be described in any one (or a combination) of the following modeling styles:
 1. **Gate-level modeling** using instantiations of predefined and user-defined primitive gates.
 2. **Dataflow modeling** using continuous assignment statements with the keyword **assign**.
 3. **Behavioral modeling** using procedural assignment statements with the keyword **always**.
- **Gate-level and Dataflow modeling** are used to model **combinatorial circuits**
- **Behavioral modeling** is used for both **combinatorial and sequential** circuits.

Gate Level/ Structural Modeling

- A Gate level description is based on the structure of the logic circuit.
- A gate level description uses the verilog primitive gates to produce the desired output
- A circuit is specified by its logic gates and their interconnections.
- Gate-level modeling provides a textual description of a schematic diagram.

Gate Level/ Structural Modeling

Verilog Primitive Gates

Function	Syntax	Expression
AND	<code>and(f, a, b, ...)</code>	$f = (a \cdot b \cdot \dots)$
OR	<code>or(f, a, b, ...)</code>	$f = (a + b + \dots)$
NOT	<code>not(f, a)</code>	$f = a'$
NAND	<code>nand(f, a, b, ...)</code>	$f = (a \cdot b \cdot \dots)'$
NOR	<code>nor(f, a, b, ...)</code>	$f = (a + b + \dots)'$
XOR	<code>xor(f, a, b, ...)</code>	$f = (a \oplus b \oplus \dots)$
XNOR	<code>xnor(f, a, b, ...)</code>	$f = (a \odot b \odot \dots)$

Gate Level/ Structural Modeling

Verilog Primitive Gates

You can also “name” the gates if you like ...

Syntax:

gate primitive_name(output, input, input...)

- and a1(out, in1, in2); // this gate is named A1
- nand n1(out, in1, in2); // instance of nand gate hardware
- xor x1(out, in1, in2); // instance of xor gate
- not n1(out, in1);
- xnor xn1(out, in1, in2);

XOR GATE: GATE LEVEL

```
module xor_gate_level(y,a,b);  
  
input a,b;  
  
output y;  
  
xor x1(y,a,b);  
  
endmodule
```

Half Adder

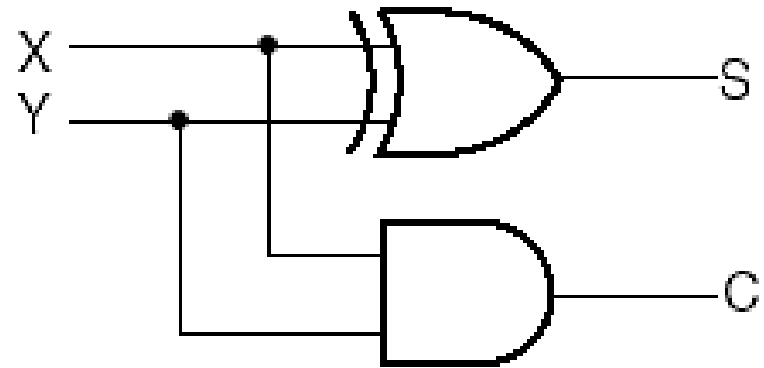
Truth Table of Half Adder

Inputs		Outputs	
X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = X \oplus Y$$

$$C = X \cdot Y$$

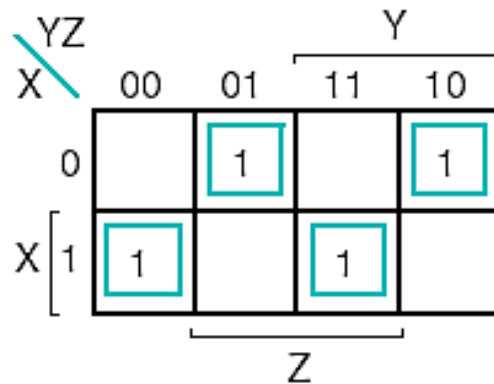
Logic Diagram of Half Adder



Full Adder

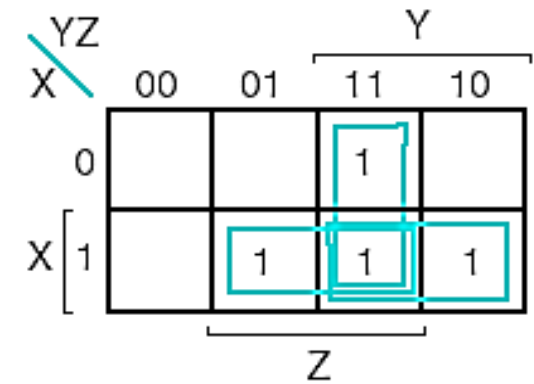
Truth Table of Full Adder

Inputs			Outputs	
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$S = \bar{X}\bar{Y}Z + \bar{X}Y\bar{Z} + X\bar{Y}\bar{Z} + XYZ$$

$$= X \oplus Y \oplus Z$$



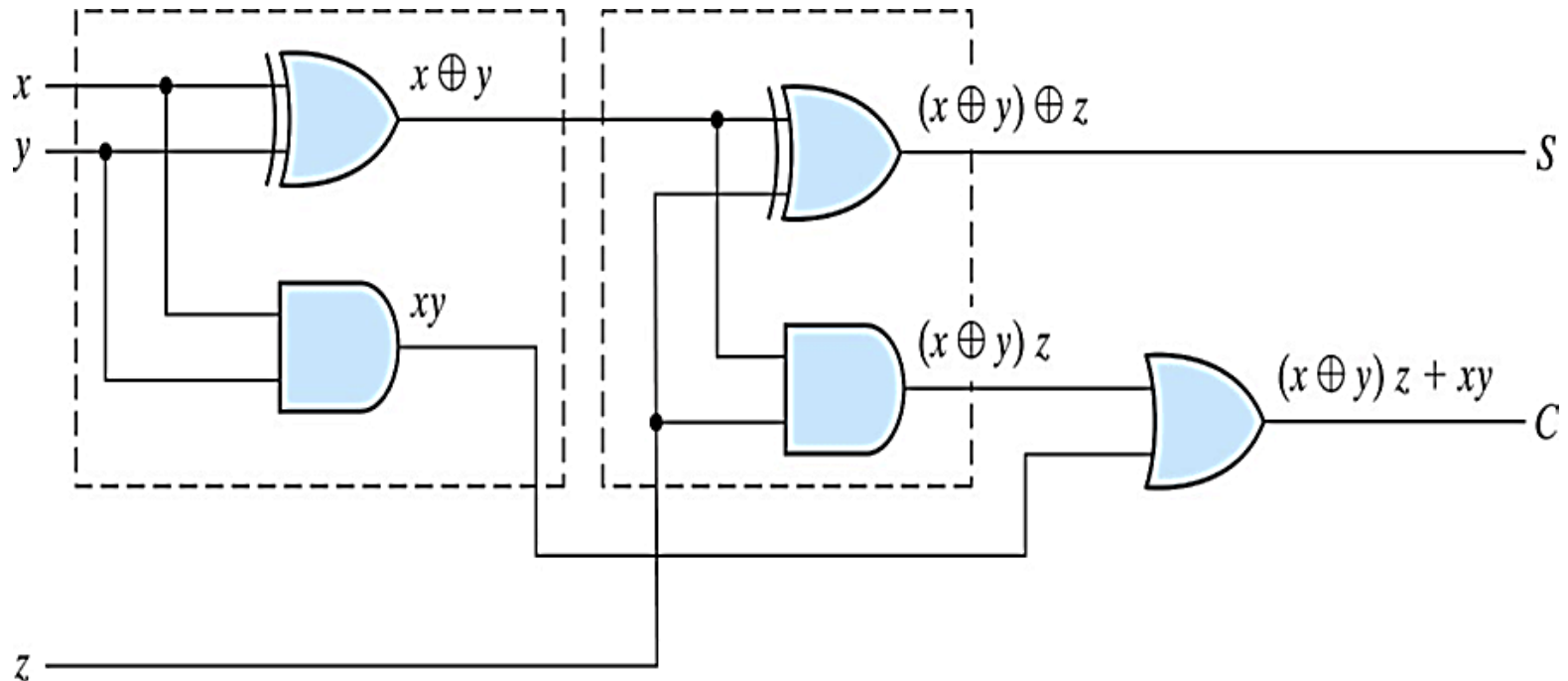
$$C = XY + XZ + YZ$$

$$= XY + Z(X\bar{Y} + \bar{X}Y)$$

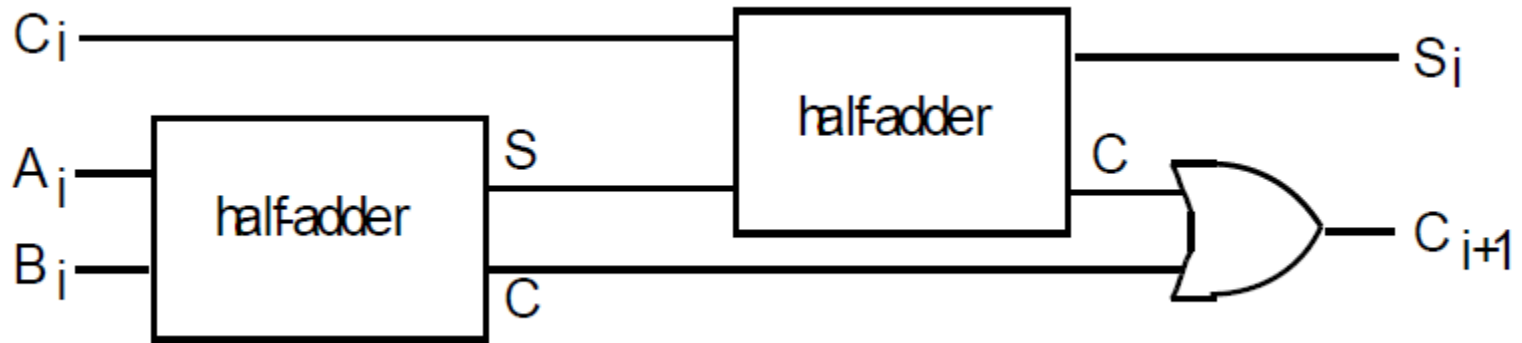
$$= XY + Z(X \oplus Y)$$

Full Adder

Logic Diagram of Full Adder



Full adder using Half adders



HALF SUBTRACTOR

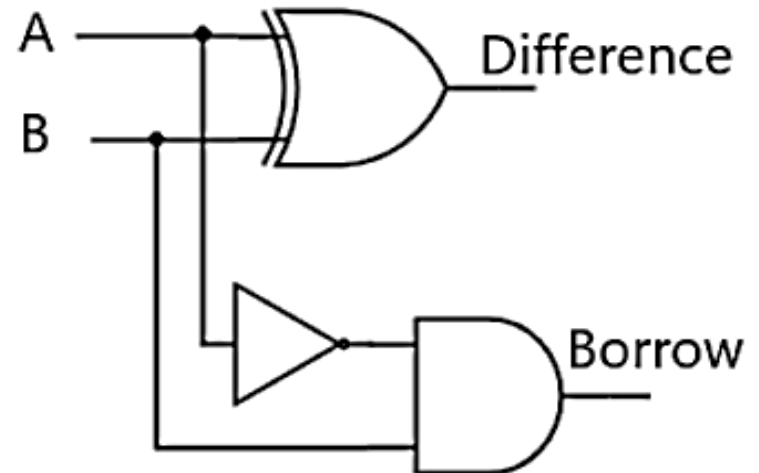
Truth Table of Half Subtractor

Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$\text{Difference} = A \oplus B$$

$$\text{Borrow} = A'B$$

Logic Diagram of Half Subtractor



FULL SUBTRACTOR

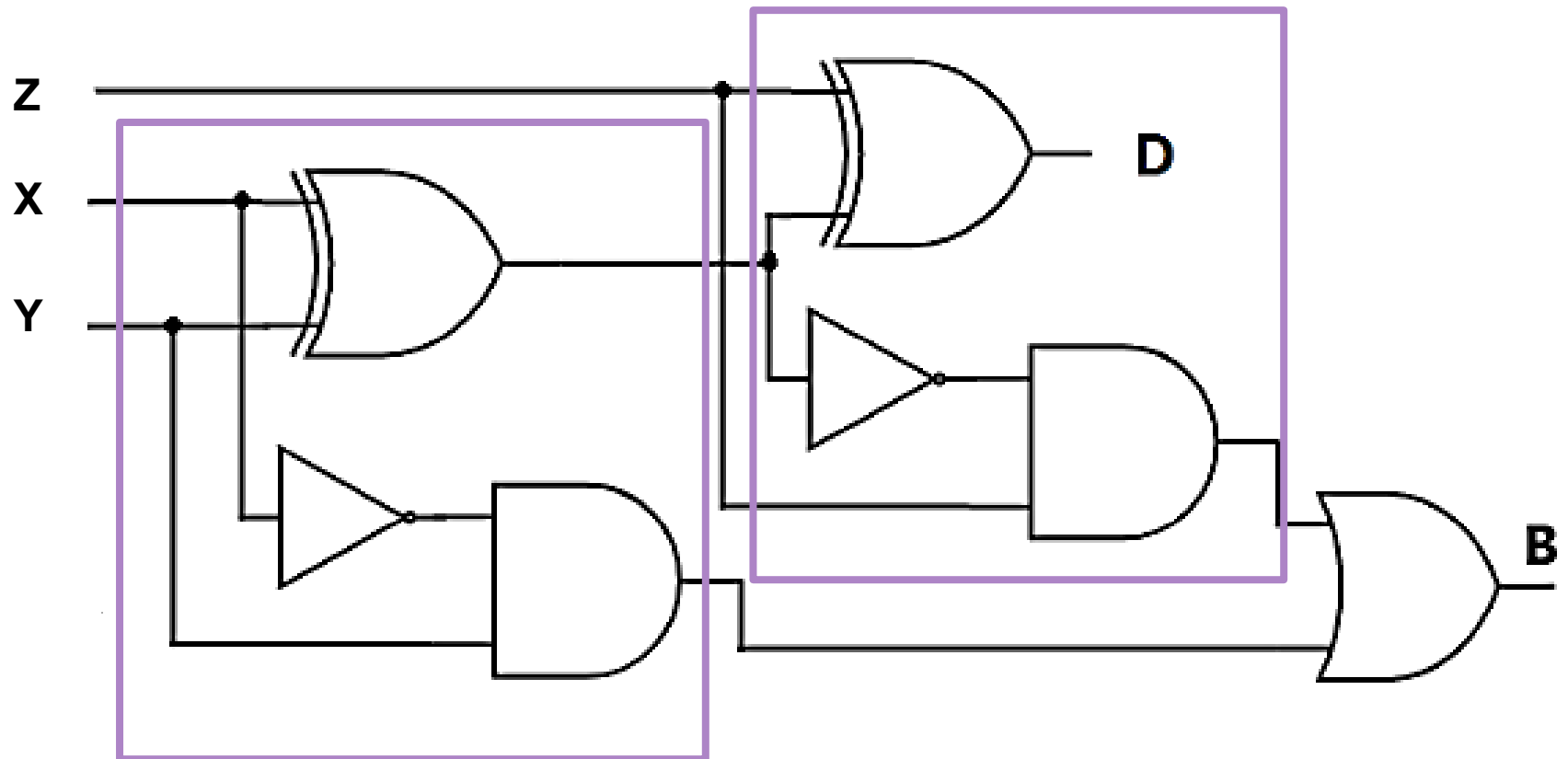
Truth Table of Full Subtractor

x	y	z	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = z \oplus (x \oplus y)$$

$$\begin{aligned} B &= z (x \oplus y)' + x'y \\ &= z(xy + x'y') + x'y \\ &= xyz + xy'z' + x'y \end{aligned}$$

Logic Diagram of Full Subtractor



Gate Level Modeling – Half Adder

//Half Adder : Structural Verilog Description

```
module ha_gate_level (x,y,s,c);
```

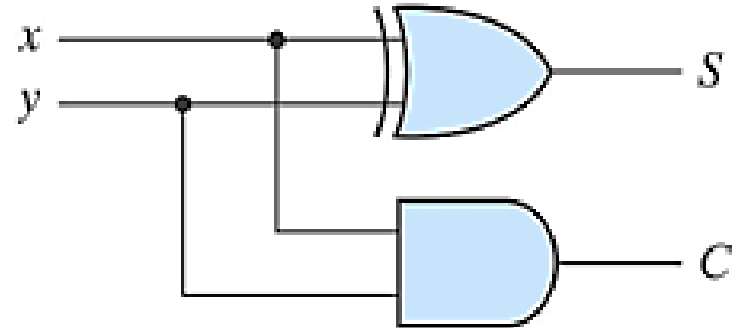
```
input x,y;
```

```
output s,c;
```

```
  xor x1(s, x, y);
```

```
  and a1(c, x, y);
```

```
endmodule
```



$$S = x \oplus y$$

$$C = x \cdot y$$

Gate Level Modeling – Full Adder

//Full Adder : Structural Description

```
module fa_gate_level(x,y,z,s,c);
```

```
  input x,y,z;
```

```
  output s,c;
```

```
  wire w1,w2,w3;
```

```
  xor xor1(w1,x,y);
```

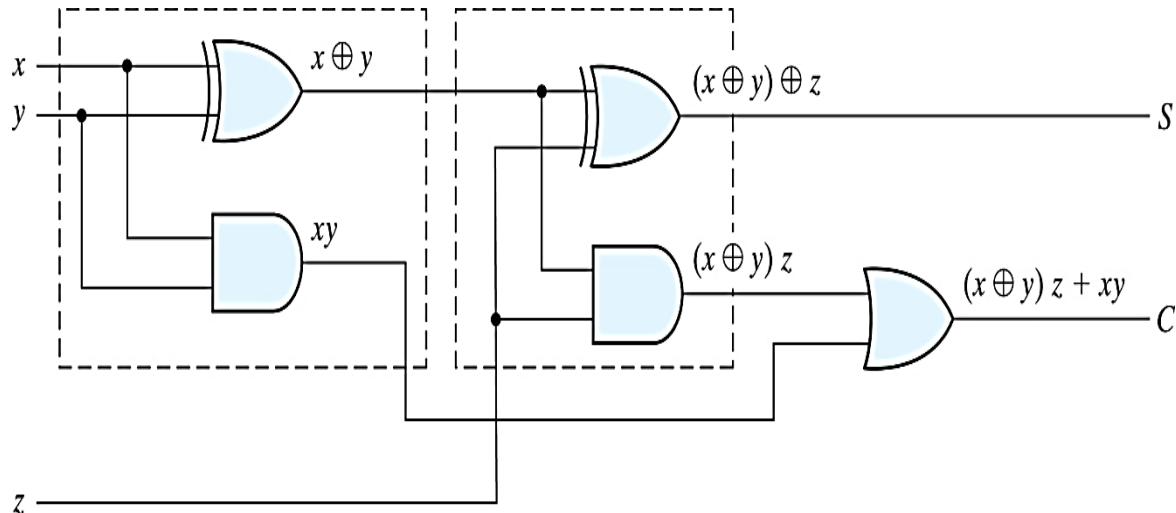
```
  xor xor2(s,w1,z);
```

```
  and A1(w2, x,y);
```

```
  and A2(w3,w1,z);
```

```
  or O1(c, w3,w2);
```

```
endmodule
```



Gate Level Modeling – Full Adder using Half adder

//Full Adder using 2 Half Adder : Structural Description

```
module fa_gate_level(x,y,z,s,c);
```

```
  input x,y,z;
```

```
  output s,c;
```

```
  wire s_ha1,c_ha1,c_ha2;
```

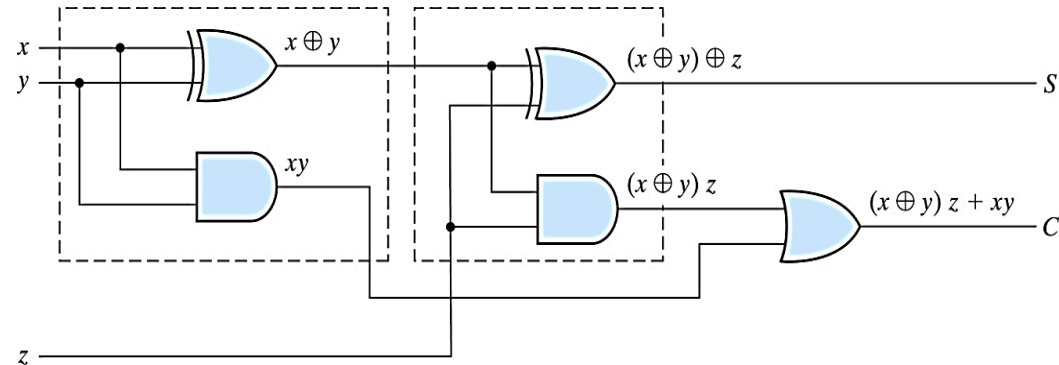
```
  //port mapping by order
```

```
  ha_gate_level HA1(x,y,s_ha1,c_ha1);
```

```
  ha_gate_level HA2(z, s_ha1, s, c_ha2);
```

```
  or O1(c, c_ha1, c_ha2);
```

```
endmodule
```



//Half Adder : Structural Verilog
Description

```
module ha_gate_level (x,y,s,c);
```

```
  input x,y;
```

```
  output s,c;
```

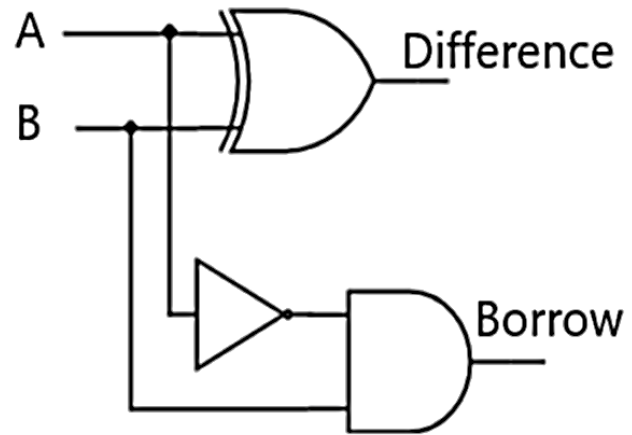
```
    xor x1(s, x, y);
```

```
    and a1(c, x, y);
```

```
endmodule
```

Gate Level Modeling – Half Subtractor

```
module half_sub_str(diff,borr,a,b);  
  input a,b;  
  output diff,borr;  
  wire w1;  
  xor x1(diff,a,b);  
  not n1(w1,a);  
  and a1(borr,w1,b);  
endmodule
```

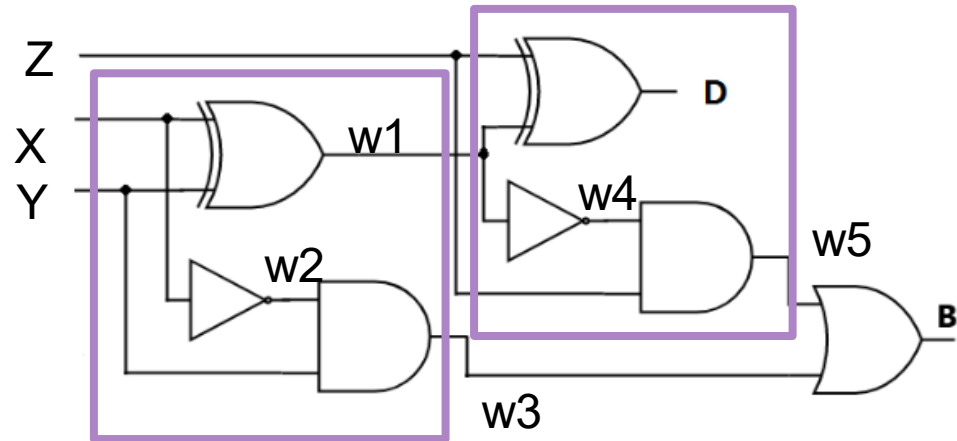


$$\text{Difference} = A \oplus B$$

$$\text{Borrow} = A'B$$

Gate Level Modeling – Full Subtractor

```
module full_sub(D,B,X,Y,Z);  
input X,Y,Z;  
output D,B;  
wire w1,w2,w3,w4,w5;  
xor x1(w1,X,Y);  
xor x2(D,w1,Z);  
not n1(w2,X);  
and a1(w3, w2,Y);  
not n2(w4,w1);  
and a2(w5, w4,Z);  
or o1(B,w5,w3);  
endmodule
```



$$D = z \oplus (x \oplus y)$$

$$\begin{aligned} B &= z (x \oplus y)' + x'y \\ &= z(xy + x'y') + x'y \\ &= xyz + xy'z' + x'y \end{aligned}$$

Dataflow Modeling

- Dataflow Modeling provides the means of describing combinational circuits by their **function** rather than by their gate structure.
- Dataflow modeling **uses a number of operators** that act on operands to produce the desired results.
- Boolean equations are used in place of logic schematics.
- Continuous assignment(**assign**) is the most basic statement used in dataflow modeling

- Dataflow modeling uses continuous assignments and the keyword assign.
- A continuous assignment is a statement that assigns a value to a net.

Verilog Code	Logic Expression	Operation
assign y = a & b	$y = a \cdot b$	AND
assign x = a b	$x = a + b$	OR
assign z = a ^ b	$z = a \oplus b$	XOR
assign z = a ~^ b or a ^~ b	$z = a \odot b$	XNOR
assign u = ~a	$u = \bar{a}$	NOT (inversion)
assign y = ~(a & b)	$y = (a \cdot b)'$	NAND

Operators

Logical

!a	not a
a && b	a and b
a b	a or b

Shift

a << b	a shift left by b
a >> b	a shift right b

Reduction

&a	reduction and
a	reduction or

Bit-Wise

~a	not a
a & b	a and b
a b	a or b
a ^ b	a xor b
a ~^ b	a xnor b
a ^~ b	

Conditional

condition ? True (exp1) : False (exp2)

Operators

Verilog HDL Operators

Symbol	Operation
+	binary addition
-	binary subtraction
&	bit-wise AND
	bit-wise OR
^	bit-wise XOR
~	bit-wise NOT
==	equality
>	greater than
<	less than
{ }	concatenation
? :	conditional

```
module small_block (a, b, c, o1, o2);
```

```
  input a, b, c;
```

```
  output o1, o2;
```

} I/O port direction declarations

```
  wire s;
```

← Internal wire (net) declarations

```
  assign o1 = s | c;
```

// OR operation

```
  assign s = a & b;
```

// AND operation

```
  assign o2 = s ^ c;
```

// XOR operation

} Logic functions

```
endmodule
```

(Keywords in bold)

Dataflow Modeling – XOR Gate

```
module xor_dataflow (f,a,b);  
    input a, b;  
    output f;  
    assign f = a^b;  
endmodule
```

Dataflow Modeling – Half Adder

//Half Adder : Dataflow description

```
module ha_df (x,y,s,c);
```

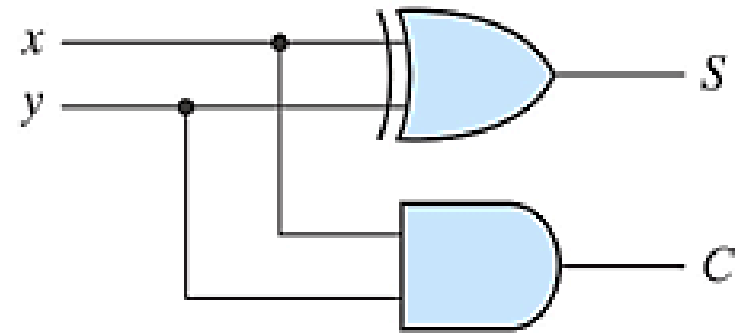
```
input x,y;
```

```
output s,c;
```

```
    assign s = x^y;
```

```
    assign c = x & y;
```

```
endmodule
```



$$S = x \oplus y$$

$$C = x \cdot y$$

Dataflow Modeling – Full Adder

//Full Adder : Dataflow description

```
module fulladder (s, c, x, y, z);
```

```
  input x, y, z;
```

```
  output s, c;
```

```
  assign s = x ^ y ^ z;  // ^ is the XOR operator in Verilog
```

```
  assign c = x & y | x & z | y & z;  // & is AND, | is OR
```

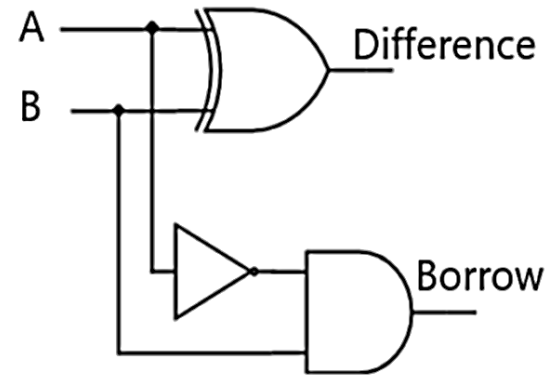
```
endmodule
```

$$S = \bar{X}\bar{Y}Z + \bar{X}Y\bar{Z} + X\bar{Y}\bar{Z} + XYZ \\ = X \oplus Y \oplus Z$$

$$C = XY + XZ + YZ \\ = XY + Z(X\bar{Y} + \bar{X}Y) \\ = XY + Z(X \oplus Y)$$

Dataflow Modeling – Half Subtractor

```
module  
half_sub_dataflow(diff,borr,a,b);  
input a,b;  
output diff,borr;  
assign diff=a^b;  
assign borr=~a&b;  
endmodule
```

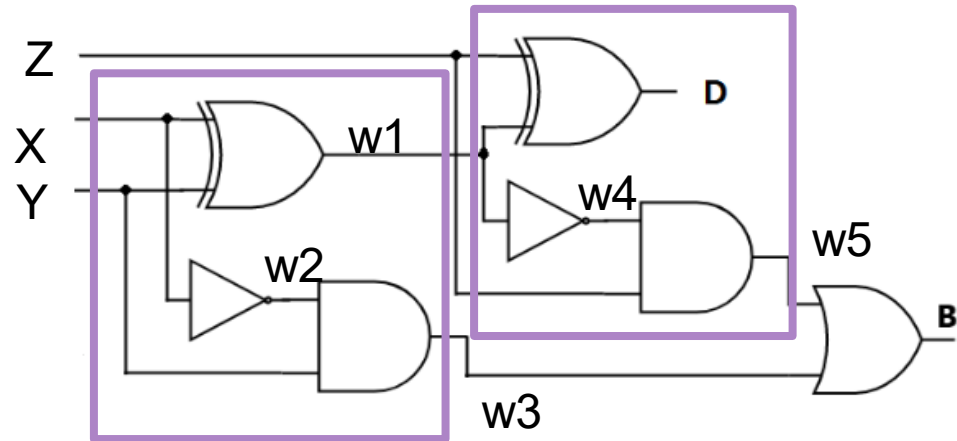


$$\text{Difference} = A \oplus B$$

$$\text{Borrow} = A' B$$

Dataflow Modeling – Full Subtractor

```
module full_sub_DL(D,B,X,Y,Z);  
input X,Y,Z;  
output D,B;  
assign D=X^Y^Z;  
assign B= ((~(X^Y))&Z) | ((~X)&Y);  
endmodule
```



$$D = z \oplus (x \oplus y)$$

$$B = z (x \oplus y)' + x'y$$

$$= z(xy + x'y')$$

$$= xyz + xy'z' + x'y$$

Full adder using 2 Half Adder

//Full Adder using 2 Half Adder : DF Description

```
module ha_df_level(s,c,x,y);
```

```
input x,y;
```

```
output s,c;
```

```
assign s=x^y;
```

```
assign c=x&y;
```

```
endmodule
```

```
module fa_df_level(s,c,x,y,z,);
```

```
input x,y,z;
```

```
output s,c;
```

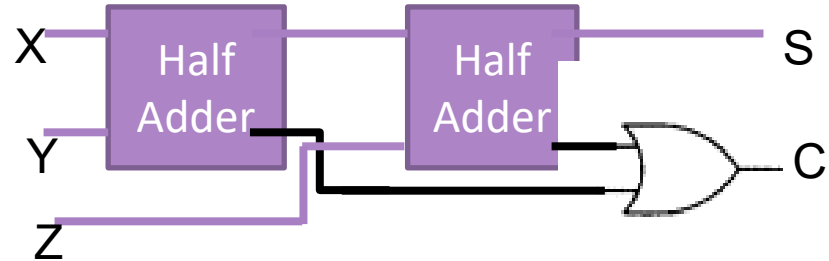
```
wire s_ha1,c_ha1,c_ha2;
```

```
ha_df_level HA1(s_ha1,c_ha1, x,y);
```

```
ha_df_level HA2(s, c_ha2 ,z, s_ha1);
```

```
assign c=c_ha1 | c_ha2;
```

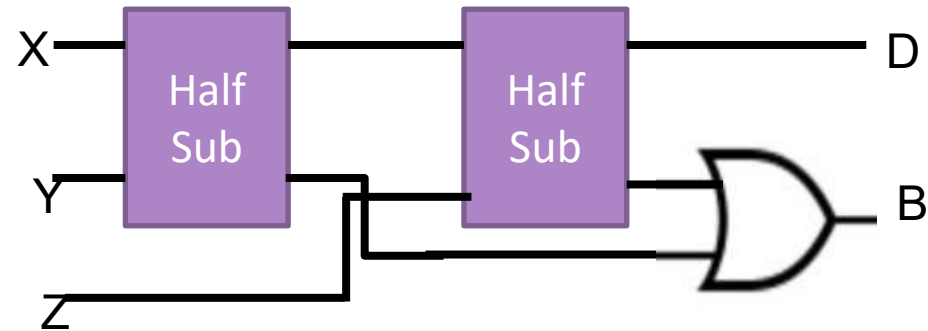
```
endmodule
```



Full Subtractor using 2 Half Subtractors

```
module HS_df_level(D,B,x,y);  
input x,y;  
output D,B;  
assign D=x^y;  
assign B=~x&y;  
Endmodule
```

```
module FS_df_level(D,B,x,y,z);  
input x,y,z;  
output D,B;  
wire D_HS1, B_HS1, B_HS2;  
HS_df_level HS1(D_HS1, B_HS1,x,y);  
HS_df_level HS2(D, B_HS2, D_HS1,z);  
assign B=B_HS1|B_HS2;  
endmodule
```



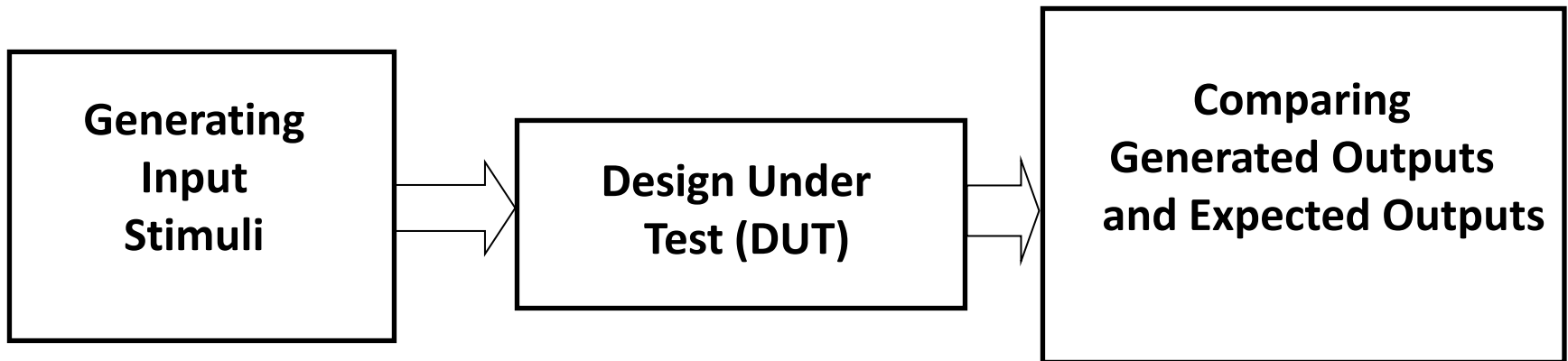
Test Bench

What is a Test Bench?

- Test Bench is a program that verifies the functional correctness of the hardware design.
- The test bench program checks whether the hardware model does what it is supposed to do.

Main Functions of a Test Bench

- Generate stimulus for testing the hardware block.
- Apply the stimulus.
- Compare the generated outputs against the expected outputs.



Design Under Test

- A design under test, abbreviated as DUT, is a **synthesizable module of the functionality we want to test**.
- Verification is required to ensure the design meets the timing and functionality requirements.
- Verilog Test benches are used to simulate and analyze designs without the need for any physical hardware or any hardware device.

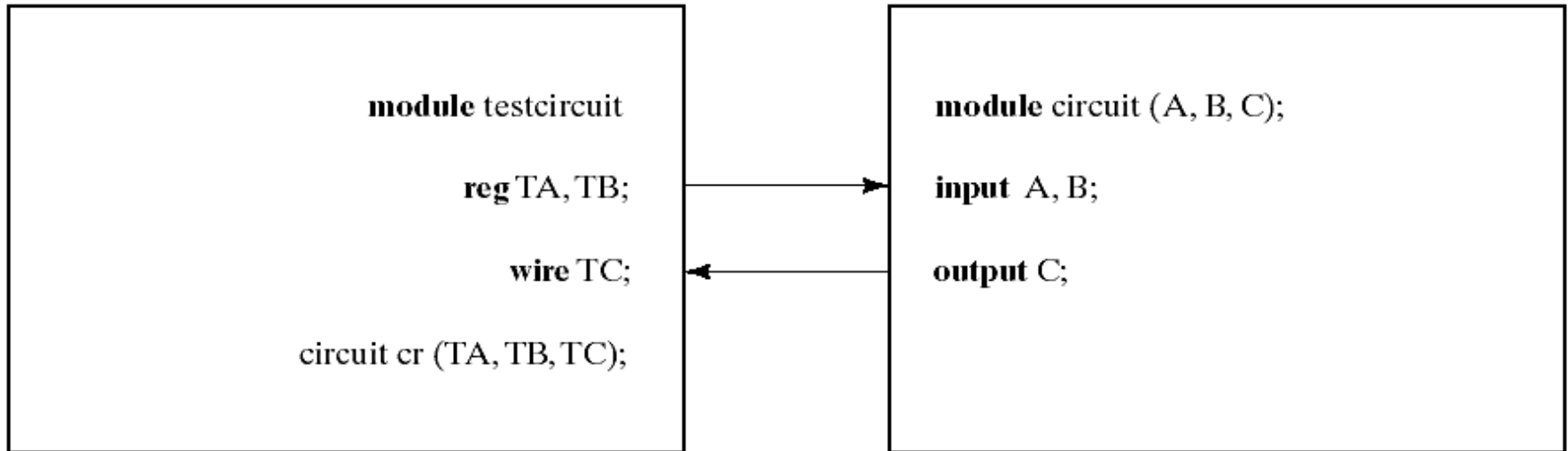
Net (wire) vs Reg

- Each signal in Verilog belongs to either a net or a register.
- A net (wire) represents a physical wire. Its signal value is determined by its driver.
- If it is not driven by any driver, its value is high impedance (Z).
- A register is like a variable in programming languages. It keeps its value until a new value is assigned to it.
- Unlike registers, nets do not have storage capacity.

Writing a Test-Bench

Stimulus module

Design module



The stimulus model generates inputs for the design module by declaring identifiers *TA* and *TB* as **reg** data type, and checks the output of the design unit with the **wire** identifier *TC*.

The local identifiers are then used to instantiate the design module under test.

Writing a Test-Bench

- The response to the stimulus generated by the **initial** and **always** blocks will appear at the output of the simulator as timing diagrams.
- It is also possible to display numerical outputs using Verilog *system tasks*.
 - **\$display** – display one-time value of variables or strings with end-of-line return,
 - **\$write** – same \$display but without going to next line.
 - **\$monitor** – display variables whenever a value changes during simulation run.
 - **\$time** – displays simulation time
 - **\$finish** – terminates the simulation
- The syntax for **\$display**, **\$write** and **\$monitor** is of the form
Task-name (format-specification, argument list);
E.g. **\$display**(%d %b %b, C,A,B);
\$display("time = %0d A = %b B=%b",**\$time**,A,B);

Typical Verilog Test Bench

```
module test_bench ;  
    reg list_of_inputs_to_DUT;  
    wire list_of_outputs_to_DUT;  
    design_under_test ( list-of-inputs-outputs-of-DUT-their-types-and-  
        modes);  
    initial  
    begin  
        Initialize-Generate-stimulus-vectors-using-behavioral-constructs;  
    end  
endmodule
```

Simple Example in Verilog

```
module and1(y,a,b);  
output y;  
input a,b;  
and a1(y,a,b);  
endmodule
```

```
module and_tb;  
reg a,b; // inputs  
wire y; // outputs  
and1 a1(.y(y), .a(a), .b(b));  
initial  
begin  
a=0; b=0; //a=1'b0; b=1'b0;  
#100 a=0; b=1;  
#100 a=1; b=0;  
#100 a=1; b=1;  
end  
endmodule
```

Test Bench - Half Adder

```
module half_add_tb;
  wire sum, carry;
  reg a, b;
  half_add h1(.sum(sum),.carry(carry),.a(a),.b(b));
  initial
  begin
    a=0;b=0;
    #100 a=0;b=1;
    #100 a=1;b=0;
    #100 a=1;b=1;
  end
endmodule
```

Test Bench - Full Adder

```
module testbench_fulladder;
    reg a, b, c;
    wire sum, cout;
    full_adder f1 (a, b, c, sum, cout);
    initial begin
        a = 1'b0; b = 1'b0; c = 1'b0;
        #20 a = 1'b0; b = 1'b0; c = 1'b1;
        #20 a = 1'b0; b = 1'b1; c = 1'b0;
        #20 a = 1'b0; b = 1'b1; c = 1'b1;
        #20 a = 1'b1; b = 1'b0; c = 1'b0;
        #20 a = 1'b1; b = 1'b0; c = 1'b1;
        #20 a = 1'b1; b = 1'b1; c = 1'b0;
        #20 a = 1'b1; b = 1'b1; c = 1'b1;
    end
endmodule
```

Test Bench - Half Subtractor

```
module half_sub_tb;
  wire diff, borr;
  reg a, b;
  half_sub h1(.diff(diff),.borr(borr),.a(a),.b(b));
  initial
  begin
    a=0;b=0;
    #100 a=0;b=1;
    #100 a=1;b=0;
    #100 a=1;b=1;
  end
endmodule
```

Test Bench - Full Subtractor

```
module full_sub_tb;
  wire diff, borr;
  reg a, b,c;
  full_sub h1(.diff(diff),.borr(borr),.a(a),.b(b),.c(c));
  initial
  begin
    a=0;b=0; c=0;
    #100 a=0;b=0; c=1;
    #100 a=0;b=1; c=0;
    #100 a=0;b=1; c=1;
    // etc... upto a=1;b=1;c=1;
  end
endmodule
```


Practice Problems

PROGRAM :

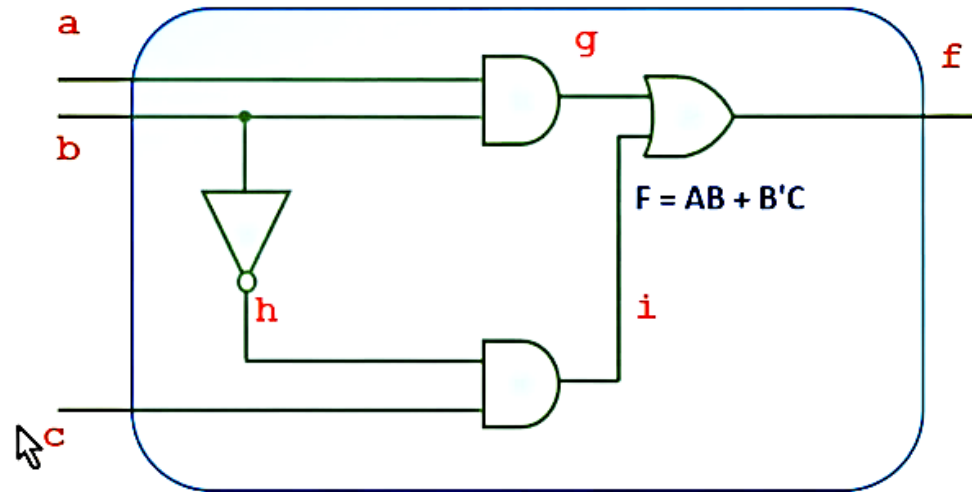
```
module basic1(a,b,YAND,YOR,YNOT,YXOR,YXNOR);  
input a,b;  
output YAND,YOR,YNOT,YXOR,YXNOR;  
assign YAND = a & b;  
assign YOR = a | b;  
assign YNOT = ~ a ;  
assign YXOR = a ^ b;  
assign YXNOR = ~(a ^ b);  
endmodule
```

TEST BENCH :

```
module basic1_tb();  
wire YAND,YOR,YNOT,YXOR,YXNOR;  
reg a,b;  
basic1 basic1_ins(a,b, YAND,YOR,YNOT,YXOR,YXNOR);  
initial  
begin  
a=0; b=0;  
#5 a=0; b=1;  
#5 a=1; b=0;  
#5 a=1; b=1;  
end  
endmodule
```

Practice Problems

```
module circuit(a,b,c,f);  
  input a,b,c;  
  output f;  
  wire g,h,i;  
  and (g,a,b);  
  not (h,b);  
  and (i,h,c);  
  or (f,g,i);  
endmodule
```



Test Bench

```
module TB_circuit ;  
  wire f1;  
  reg a1, b1, c1;  
  circuit my_module(a1, b1, c1, f1);  
  initial  
  begin  
    a1 = 1'b0; b1 = 1'b1; c1 = 1'b0;  
  
    #5  
    a1 = 1'b1; b1 = 1'b1; c1 = 1'b1;  
  
    #5  
    a1 = 1'b1; b1 = 1'b0; c1 = 1'b1;  
  
    #5  
    a1 = 1'b1; b1 = 1'b1; c1 = 1'b1;  
  end  
endmodule
```

- No port declaration is required
- my_module is name of instance
- Instantiated module (port sequence should not change)
- 1 bit binary number
- Only inputs to the DUT are changing (a1,b1,c1).
- f1 is the output from the DUT to test bench (which we monitor)